

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
PAULISTA

Engenharia de Software

Cainã Sandes Batista

RM: 568571

GS – COMPUTATIONAL THINKING WITH PYTHON

Funcionalidades da solução

São Paulo

2026

Cainã Sandes Batista

RM: 568571

GS – COMPUTATIONAL THINKING WITH PYTHON

Funcionalidades da solução

Global Solution apresentada ao curso de Engenharia de Software da FIAP, orientada pelo Prof. Edson de Oliveira.

São Paulo

2026

SUMÁRIO

DESCRIÇÃO DO PROJETO	3
FUNCIONALIDADES DO SISTEMA.....	4
REQUISITOS E DEPENDÊNCIAS.....	4
INSTRUÇÕES DE USO	5
CONCEITOS DE PYTHON APLICADOS	5
PARÂMETROS DE TELEMETRIA	6
REFERÊNCIAS.....	7

DESCRIÇÃO DO PROJETO

O Hermes-1 é um ecossistema integrado de monitoramento de suporte à vida e telemetria para cápsulas espaciais, simulando os parâmetros críticos de habitabilidade monitorados durante missões em órbita baixa (LEO). A solução abrange desde a coleta de dados via hardware embarcado (ESP32) e a orquestração de fluxo no Node-RED, até à visualização em tempo real em um painel de controle em React e o sistema de auditoria desenvolvido em Python. Por automatizar infraestruturas críticas e viabilizar a segurança de tripulações que operam satélites de monitoramento climático, o projeto está diretamente conectado aos ODS 9 (Indústria, Inovação e Infraestrutura) e ODS 13 (Ação Contra a Mudança Global do Clima).

Este módulo específico, o módulo de auditoria pós-voo, consiste no script em Python responsável pelo ecossistema de inteligência terrestre da missão. Ele simula o comportamento de um software de diagnóstico na base de controle, processando e analisando o histórico de logs gerados pelo veículo após o voo para identificar anomalias nos parâmetros críticos de habitabilidade.

FUNCIONALIDADES DO SISTEMA

O sistema é operado por um menu principal com cinco opções:

- **Opção 1 - Criar log de exemplo:** Gera o arquivo `telemetria_flight.txt` com dados simulados de telemetria da missão, incluindo leituras nominais, leituras críticas e linhas intencionalmente corrompidas para fins de teste do tratamento de exceções;
- **Opção 2 - Carregar log:** Lê e processa o arquivo de telemetria linha por linha, validando cada registro e ignorando entradas inválidas com o devido aviso em tela ao usuário;
- **Opção 3 - Diagnóstico da missão:** Exibe as médias de oxigênio, pressão e temperatura, a quantidade de eventos críticos registrados e o detalhamento de cada anomalia detectada com seu respectivo *timestamp*;
- **Opção 4 - Calcular autonomia de O2:** Recebe a reserva atual de oxigênio em litros e o consumo médio por hora, calcula o tempo estimado de suporte à vida e classifica a situação como crítica, limitada ou dentro do esperado;
- **Opção 5 - Gerar relatório:** Salva o relatório completo de auditoria em um arquivo.txt com todos os dados processados da missão, incluindo as médias, anomalias e informações da tripulação.

REQUISITOS E DEPENDÊNCIAS

Requisitos de sistema:

- Python 3.10 ou superior (necessário para uso da estrutura match/case);
- Sistema Operacional: Windows, macOS ou Linux.

Dependências:

- Nenhuma biblioteca externa é necessária. O único módulo utilizado é o `os`, nativo do Python, responsável pela limpeza do terminal.

Arquivos do projeto:

- `Hermes-1.py`: Arquivo principal do script, que deve ser executado diretamente;
- `telemetria_flight.txt`: Arquivo de log gerado automaticamente pela opção 1 do menu;
- `relatorio_missao.txt`: Relatório gerado automaticamente pela opção 5 do menu.

INSTRUÇÕES DE USO

Para executar o projeto, siga os passos abaixo:

- Certifique-se de ter o Python 3.10 ou superior devidamente instalado na máquina;
- Salve o arquivo `Hermes-1.py` em uma pasta de sua preferência;
- Abra o terminal do sistema operacional e navegue até à pasta onde o arquivo foi salvo;
- Execute o script utilizando o comando: `python Hermes-1.py`;
- No menu principal, selecione a **Opção 1** para criar o arquivo de log de exemplo;
- Em seguida, selecione a **Opção 2** para carregar os dados do log na memória do sistema;
- Explore as demais opções disponíveis para exibição de diagnóstico, cálculo de autonomia e geração de relatório.

CONCEITOS DE PYTHON APLICADOS

O script foi desenvolvido cobrindo todos os requisitos técnicos obrigatórios da disciplina:

- **Variáveis e tipos de dados:** Utilizados em todo o script para armazenar configurações, contadores, flags de controle e resultados de cálculos matemáticos;
- **Estruturas de controle (if/elif/else):** Presentes na verificação de limites críticos em `detectar_anomalias()`, na validação de entradas em `menu_autonomia()` e na classificação de segurança da autonomia de oxigênio;

- **Estruturas de repetição (while e for):** O laço *while* controla o *loop* principal do menu e a navegação do sistema. O laço *for* é utilizado para iterar sobre os registros nas funções de análise, na leitura do log e na escrita do relatório;
- **Listas:** Utilizadas na variável *registros_carregados* (para manter os dados válidos na memória), no armazenamento das linhas lidas do arquivo, no retorno das anomalias detectadas e na listagem de tripulantes da missão;
- **Manipulação de strings:** Uso de *strip()* para remover espaços e quebras de linha residuais, *split(';')* para realizar o *parsing* dos campos do log, *join()* para formatar listas em texto legível e *f-strings* para a montagem dinâmica das mensagens de saída;
- **Tuplas:** A variável *LIMITES_CRITICOS* é uma tupla contendo os três limites de segurança da cabine. O uso dessa estrutura é intencional, dado que os limites de segurança são valores imutáveis por definição de projeto;
- **Dicionários:** A estrutura *INFO_MISSAO* armazena as informações fixas da missão. Adicionalmente, cada registro de telemetria é processado e estruturado como um dicionário contendo as chaves *timestamp*, *o2*, *pressao*, *temperatura* e *status*. As médias calculadas e as anomalias também retornam em formato de dicionário;
- **Funções com parâmetros e retorno:** Implementação de funções modulares como *calcular_medias(registros)*, *filtrar_criticos(registros)*, *detectar_anomalias(registros)* e *calcular_autonomia(reserva, consumo)*, todas aplicando passagem de parâmetros e retornos bem definidos;
- **Manipulação de arquivos:** Leitura do arquivo de entrada *telemetria_flight.txt* com *open()* no modo 'r' e escrita do relatório de saída *relatorio_missao.txt* com *open()* no modo 'w', ambos utilizando o gerenciador de contexto *with* e codificação UTF-8;
- **Tratamento de exceções:** Uso de blocos *try/except* para capturar falhas de forma robusta, como *FileNotFoundError* para arquivo ausente, *ValueError* para campos não numéricos convertidos incorretamente, *IndexError* para linhas corrompidas com campos faltando, além de cláusulas genéricas para precaver erros inesperados de E/S (Entrada/Saída);
- **match/case:** Estrutura utilizada no menu principal para controle de fluxo e navegação limpa entre as opções do sistema.

PARÂMETROS DE TELEMETRIA

Os três parâmetros monitorados pelo sistema refletem métricas reais aplicadas em sistemas de suporte à vida aeroespaciais:

- **Concentração de oxigênio (O2):** Mensurado em porcentagem (%). Faixa nominal: 20.9%. Limite crítico de segurança: abaixo de 19.5%. (*Origem no ecossistema: Potenciômetro 1 no ESP32*);

- **Pressão atmosférica da cabine:** Mensurado em quilopascals (kPa). Faixa nominal: 101.3 kPa. Limite crítico de segurança: abaixo de 95.0 kPa. (*Origem no ecossistema: Potenciômetro 2 no ESP32*);
- **Temperatura interna da cabine:** Mensurado em graus Celsius (°C). Faixa nominal: 22.5 °C. Limite crítico de segurança: acima de 28.0 °C. (*Origem no ecossistema: Sensor DHT22 no ESP32*).

Estes mesmos limites de consistência estão mapeados de forma síncrona na tupla *LIMITES_CRITICOS* do script Python, no código-fonte C++ do microcontrolador ESP32 e no arquivo JSON estruturado que alimenta o painel gráfico em React, assegurando a integridade de dados por completo no ecossistema Hermes-1.

REFERÊNCIAS

DE OLIVEIRA, Edson. 1ES Ago - Global Solution 2026, computational thinking with python. May 2026. Disponibilizado em pdf.

SANDES, Cainã. Hermes-1. Jun 2026. Disponível em:

<https://github.com/Sednas22/Hermes-1/tree/ComputationalThinkingWithPython>